



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Algo Verse: An Interactive Platform for Visualizing, Comparing, and Learning Algorithms

A CAPSTONE PROJECT REPORT

A Capstone Project Report submitted in partial fulfilment of the requirements for the Course of

CSA0609-DESIGN AND ANALYSIS OF ALGORITHMS

to the award of the degree of

**BACHELOR OF ENGINEERING IN
ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING**

Submitted by

Paleru Dharani Goavrdhan(19525280)

Under the Supervision of

Dr P Solainayagi

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences Chennai-602105

SEPTEMBER 2026



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled "**Algo Verse: An Interactive Platform for Visualizing, Comparing, and Learning Algorithms**" has been carried out by Paleru Dharani Govardhan (192525280) part of a five-member project team under the supervision of **Dr. Solainayagi P** and is submitted in partial fulfilment of the requirements for the current semester of the B-TECH Artificial Intelligence and Machine Learning program at Saveetha Institute of Medical and Technical Sciences, Chennai.

Course Coordinator

Dr. _____

Dept. of CSE

**SIMATS Engineering,
SIMATS, Chennai-602105.**

Course Faculty

Dr.Solainayagi P

Dept. of CSE

**SIMATS Engineering,
SIMATS, Chennai-602105.**



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

DECLARATION

I, G Durga Surya Prasad (12425158) , SIMATS ENGINEERING, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled "**Algo Verse: An Interactive Platform for Visualizing, Comparing, and Learning Algorithms**" is the result of the bonafide efforts of our project team. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date: 03/09/2026

Signature of the Students with Names

K. Hemanth (192572074)

G. Durga Surya Prasad (192425158)

P. Dharani Govardhan (192525280)

N. Kameshwar Reddy (192525272)

P. Manoj (192411081)



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Individual Contribution Statement

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
K. Hemanth (192572074)	Module 1 -- Algorithm Visualizer	Designed and implemented the step-by-step visualization engine and animation controller for sorting, searching, graph and DP algorithms.	Tested visualizer rendering across input sizes and browsers; validated step accuracy.	Prepared visualizer design methodology and implementation documentation.	20%
G. Durga Surya Prasad (192425158)	Module 2 -- Algorithm Comparator	Designed and implemented the multi-algorithm comparator engine that runs	Tested comparator accuracy and timing consistency across	Prepared comparator architecture, methodology and results documentation.	20%

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
		algorithms on identical input and captures step/time metrics.	repeated runs.		
P. Dharani Govardhan (192525280)	Module 3 -- Real-World Problem Solver	Implemented the problem-to-algorithm mapping engine and curated real-world problem scenario dataset.	Tested mapping accuracy against curated problem-algorithm pairs.	Prepared problem-mapping logic and dataset documentation .	20%
N. Kameshwar Reddy (192525272)	Module 4 -- Complexity Analyzer	Implemented theoretical Big-O lookup and empirical runtime/space benchmarking and curve-	Evaluated empirical vs theoretical complexity alignment across algorithms.	Prepared complexity-analysis methodology and results documentation .	20%

Student Name / Register No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
		plotting logic.			
P. Manoj (192411081)	Overall System Integration & Final Report	Integrated all four modules into a single Flask/Node-based web application and unified UI.	Performed complete system integration testing across modules.	Consolidated the overall project report.	20%
Total	All Modules	Complete system design	Complete system testing	Complete project report	100%

ABSTRACT

Learning data structures and algorithms (DSA) is a foundational requirement in computer science education, yet most learners are introduced to algorithms through static pseudocode, textbook diagrams, or pre-recorded video lectures. These formats convey the final logic of an algorithm but rarely reveal how the algorithm behaves step by step, how it compares against alternative approaches on the same input, how it maps onto a practical real-world scenario, or how its theoretical complexity actually manifests as measurable running time. As a result, many learners can recite an algorithm's pseudocode without developing an intuitive understanding of its behaviour, efficiency, or applicability.

This project presents "AlgoVerse: An Interactive Platform for Visualizing, Comparing, and Learning Algorithms", an interactive, web-based learning platform designed to make algorithm education visual, comparative, practical, and analytical. The platform is organized into four integrated modules: an Algorithm Visualizer that animates the step-by-step execution of sorting, searching, graph, and dynamic-programming algorithms; an Algorithm Comparator that executes multiple algorithms on the same input simultaneously and reports step counts and execution time side by side; a Real-World Problem Solver that accepts a description of a practical problem scenario and recommends suitable algorithm(s) with reasoning; and a Complexity Analyzer that presents theoretical Big-O time and space complexity alongside empirically measured runtime and growth curves.

The methodology includes curating an algorithm reference dataset and a real-world problem-scenario dataset, building a browser-based visualization and execution engine, implementing comparator and complexity-benchmarking logic, and integrating all modules behind a unified, responsive web dashboard. The resulting prototype allows a learner to select or input an algorithm, watch its execution unfold visually, compare it against alternative algorithms on identical data, see which real-world problems it is best suited to, and view how its measured performance aligns with its theoretical complexity class.

The major outcome of the project is a functional, extensible prototype that unifies visualization, comparison, practical-problem mapping, and complexity analysis in a single platform, rather than requiring learners to rely on multiple disconnected tools. The project demonstrates that combining these four perspectives significantly improves the interpretability of algorithm behaviour for

learners, while further work on dataset breadth, personalization, and classroom-scale deployment is recommended before wide adoption.

Keywords: Algorithm Visualization, Algorithm Comparison, Complexity Analysis, Data Structures and Algorithms Education, Interactive Learning, Big-O Notation, Web-Based Learning Tools.

TABLE OF CONTENTS

Sl.No	Title	Page No
i	CERTIFICATE FROM THE SUPERVISOR	ii
ii	DECLARATION BY THE CANDIDATES	iii
iii	INDIVIDUAL CONTRIBUTION STATEMENT	iv
iv	ABSTRACT	v
v	LIST OF FIGURES	vi
vi	LIST OF TABLES	vii
vii	LIST OF ABBREVIATIONS	viii
1	CHAPTER 1: Introduction	8-12
2	CHAPTER 2: Literature Review	13-17
3	CHAPTER 3: Engineering Design & Trade-offs, Sustainability, Ethics, Safety, Risk & Security	18-24
4	CHAPTER 4: Implementation, Testing & Results, Project Management	25-31
5	CHAPTER 5: Conclusion & Reflection	32-34
	References	34-35
	Appendices	35-42

LIST OF FIGURES

Figure No.	Figure Name
1	System Architecture Flow
2	Comparator Runtime Comparison Chart
3	Complexity Analyzer Growth Curve (Empirical vs Theoretical)

LIST OF TABLES

Table No.	Table Name
1	Stakeholder / User Requirements
2	Functional & Non-Functional Requirements
3	Project Constraints
4	Applicable Standards / Codes
5	Design Specifications
6	Evaluation Matrix
7	Trade-off Analysis
8	Sustainability, Ethics, Safety, Risk & Security
9	Risk Register
10	Materials, Components and Resources
11	Tools Used
12	Test Plan & Verification
13	Sample Output
14	Achievement of Objectives
15	Project Milestone Timeline
16	Roles / Contribution
17	Project Budget

LIST OF ABBREVIATIONS

Abbreviation	Full Form
DSA	Data Structures and Algorithms
UI	User Interface
UX	User Experience
API	Application Programming Interface
DP	Dynamic Programming
FPS	Frames Per Second
WCAG	Web Content Accessibility Guidelines

CHAPTER 1

INTRODUCTION AND ENGINEERING PROBLEM

Background

Algorithms are the conceptual core of computer science, yet they are inherently abstract: an algorithm is a sequence of state changes that unfolds over time, and this dynamic behaviour is difficult to convey through static pseudocode printed in a textbook or a slide. Learners are commonly taught to trace algorithms by hand on paper, watch pre-recorded video walkthroughs, or read line-by-line pseudocode, none of which adapt to the learner's own input data or allow direct comparison between competing approaches. Consequently, many students can memorize an algorithm's steps without truly internalizing why one algorithm outperforms another on a given input, or which algorithm is the right engineering choice for a given real-world problem.

With the maturity of modern browser-based visualization technologies (such as scalable vector graphics and interactive charting libraries) it has become practical to build tools that animate algorithm execution step by step, run multiple algorithms on identical data to compare them directly, connect abstract algorithms to concrete real-world problem scenarios, and plot measured execution behaviour against theoretical complexity classes. This project, AlgoVerse, develops such an integrated platform to make algorithm learning visual, comparative, practical, and analytically grounded.

Need for the Project

The project is needed because:

- Static pseudocode and textbook diagrams do not convey the dynamic, step-by-step behaviour of algorithms.
- Learners rarely get to directly compare two or more algorithms on the same input to see efficiency differences.
- Many students struggle to connect abstract algorithms with the real-world problems they are meant to solve.
- Theoretical Big-O notation is often taught in isolation from measurable, empirical runtime behaviour.

- Existing visualization tools are typically single-purpose and do not integrate comparison or complexity analysis.
- Interactive, self-paced tools can supplement classroom teaching and support independent learners and bootcamp students.
- Clear visual and analytical feedback makes algorithm learning outcomes easier to assess and retain.

People and Organizations Affected

- Undergraduate and postgraduate computer science students
- Faculty and instructors teaching Data Structures and Algorithms courses
- Coding bootcamps and online training platforms
- Self-learners and competitive-programming aspirants
- Curriculum designers and educational-technology teams
- System administrators maintaining the learning platform

Engineering Significance

The project combines Data Structures and Algorithms, Web Application Development, Data Visualization, and Human-Computer Interaction to create an engineering solution for algorithm education, integrating visual, comparative, applied, and analytical learning perspectives into a single accessible platform.

Problem Statement

The engineering problem is to design and develop an interactive web-based platform that can visually animate the step-by-step execution of common algorithms, compare multiple algorithms on identical input, recommend suitable algorithms for real-world problem scenarios, and analyze theoretical versus empirical algorithmic complexity, despite variability in input size, algorithm category, and browser rendering performance.

The system must provide an intuitive, responsive learning experience while operating under practical constraints such as in-browser computational limits, a curated (not exhaustive) algorithm

library, and the need for a simple, accessible interface usable by learners of varying experience levels. In addition to visualization, the system should provide comparative and analytical insight so that learners can reason about algorithm trade-offs, not just observe them.

Project Aim

The main aim of this project is to design and develop an interactive web-based platform, AlgoVerse, that visualizes, compares, and analyzes algorithms, and maps them to real-world problem scenarios, in order to support deeper and more intuitive algorithm learning.

Project Objectives

- To design an integrated system for interactive, browser-based algorithm learning.
- To curate an algorithm reference dataset covering sorting, searching, graph, and dynamic-programming algorithms.
- To design and implement an Algorithm Visualizer that animates step-by-step algorithm execution.
- To design and implement an Algorithm Comparator that runs multiple algorithms on identical input and reports comparative metrics.
- To curate a real-world problem-scenario dataset and implement a Real-World Problem Solver that recommends suitable algorithms.
- To implement a Complexity Analyzer that presents theoretical Big-O complexity alongside empirically measured runtime and space behaviour.
- To develop a unified, responsive web dashboard integrating all four modules.
- To evaluate platform functionality and usability through structured testing.
- To test and validate the complete system using a range of algorithms, input sizes, and problem scenarios.
- To generate reports summarizing comparison results and complexity analysis for learners.

Scope

Included

- A curated algorithm library covering sorting, searching, graph, and dynamic-programming algorithms.
- Step-by-step visual animation of algorithm execution (Algorithm Visualizer).
- Side-by-side execution and metric comparison of multiple algorithms (Algorithm Comparator).
- Mapping of real-world problem scenarios to recommended algorithms with reasoning (Real-World Problem Solver).
- Theoretical and empirical complexity analysis with growth-curve visualization (Complexity Analyzer).
- A unified, responsive web-based dashboard and history/bookmark logging.

Excluded

- Execution or visualization of arbitrary user-submitted source code beyond the curated algorithm library.
- Formal, large-scale classroom deployment and learning-management-system integration.
- Support for extremely large datasets (e.g., millions of elements) requiring server-side distributed execution.
- Guaranteed personalized learning-path recommendation based on individual learner history.
- Complete replacement of formal instructor-led DSA coursework.

Assumptions

- Learners access the platform through a modern desktop or tablet web browser.
- Input sizes used for visualization and comparison remain within practical in-browser rendering limits.
- The curated algorithm and problem-scenario datasets are representative of common introductory and intermediate DSA topics.

Boundary Conditions

- Visualization smoothness depends on input size, algorithm complexity class, and client device performance.

- Empirical timing measurements can vary with browser load and hardware, so results are indicative rather than absolute.
- The system provides AI/rule-assisted algorithm recommendations, not a definitive engineering decision for production systems.

Expected Engineering Outcome

The expected outcome is a functional, web-based algorithm-learning prototype consisting of four integrated modules delivered through a single dashboard.

The system is expected to:

- Accept a learner's choice of algorithm and input data (or generate sample data).
- Animate the step-by-step execution of the selected algorithm.
- Run and compare multiple algorithms on identical input, reporting step counts and execution time.
- Accept a description of a real-world problem scenario and recommend suitable algorithm(s) with reasoning.
- Display theoretical Big-O time/space complexity alongside empirically measured performance curves.
- Maintain a history of previous visualizations, comparisons, and analyses.
- Present all results through a clear, responsive, and accessible web interface.

Overall Engineering Outcome

Algorithm / Problem Input → Visualization Engine → Comparator Engine → Problem-to-Algorithm Mapping → Complexity Analysis (Theoretical & Empirical) → Unified Dashboard & History

The final prototype demonstrates the practical application of interactive visualization and comparative analysis to support intuitive, evidence-based algorithm learning.

CHAPTER 2

LITERATURE REVIEW AND EXISTING SOLUTIONS

Review Method

The literature review was conducted to understand existing techniques and tools used for teaching and visualizing algorithms, with particular focus on interactive visualization, algorithm comparison, and complexity-analysis pedagogy. Relevant resources were identified using search terms such as algorithm visualization, data structure animation tools, algorithm comparison learning tools, complexity analysis education, and interactive computer science education platforms.

The review considered traditional teaching methods (textbooks and pseudocode), classroom and MOOC video-based instruction, existing single-purpose visualization tools, and competitive-programming platforms, with particular attention to how each approach conveys step-by-step behaviour, comparative efficiency, and complexity understanding. Existing literature indicates that visualization improves comprehension of algorithm behaviour, but that most existing tools remain single-purpose and are not integrated with comparative or complexity-analysis features.

Review of Existing Work

Textbook and Pseudocode-Based Teaching

Traditional DSA instruction relies heavily on textbooks such as Introduction to Algorithms and Algorithms, which present pseudocode, proofs, and complexity derivations in static form, supplemented by hand-traced examples on paper or whiteboard.

Advantages:

- Rigorous and mathematically precise treatment of algorithm correctness and complexity.
- Widely available and foundational to formal computer science curricula.

Limitations:

- Does not show dynamic, step-by-step execution behaviour.
- Requires significant learner effort to mentally simulate algorithm state changes.

- Provides no direct way to compare multiple algorithms on the same input.

Existing Visualization and Learning Platforms

Dedicated algorithm-visualization websites, such as VisuAlgo and university-hosted data-structure visualization tools, animate the execution of individual algorithms and have been widely adopted in introductory computer science courses. Open-source algorithm-visualizer projects extend this idea to a broader community-maintained catalog of animated algorithms. MOOC platforms similarly offer recorded video walkthroughs of algorithm behaviour as part of structured online courses, while competitive-programming judges focus primarily on correctness and runtime scoring of submitted code rather than step-by-step visual explanation.

These tools consistently demonstrate that visual, step-by-step animation improves learner comprehension of algorithm mechanics compared to static pseudocode alone. However, most such tools are single-purpose visualizers: they do not provide side-by-side comparison of multiple algorithms on the same input, do not map algorithms to real-world problem scenarios, and typically present theoretical complexity as a label rather than connecting it to measured, empirical performance.

Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Textbook / Pseudocode Teaching	Rigorous, foundational, widely available	Static, requires mental simulation, no direct comparison	Provides motivation for interactive visualization
MOOC Video Walkthroughs	Structured, guided explanation	Not interactive, fixed pace, no user-driven input	Motivates self-paced, interactive alternative

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Single-Purpose Visualization Tools (e.g., VisuAlgo-style)	Clear step-by-step animation of individual algorithms	Typically single-algorithm focus, no built-in comparator or complexity analytics	Basis for the Visualizer module in this project
Competitive Programming Judges	Rigorous correctness/runtime scoring on real submissions	Focused on scoring, not visual explanation or comparison	Motivates empirical timing approach in Complexity Analyzer
Proposed System (AlgoVerse)	Visualizer + Comparator + Problem Mapping + Complexity Analysis in one platform	Requires curated dataset and further real-classroom validation	Integrated solution addressing identified gaps

Research / Engineering Gap

The literature indicates that several challenges remain unresolved:

- Most existing visualization tools focus on a single algorithm at a time, without built-in side-by-side comparison.
- Few tools explicitly connect abstract algorithms to concrete, real-world problem scenarios.
- Theoretical Big-O complexity is often taught separately from measurable, empirical runtime behaviour.
- Learners frequently must use multiple disconnected tools (a visualizer, a complexity cheat-sheet, and a coding judge) to build a complete understanding.
- There is limited integration of visualization, comparison, applied problem-mapping, and complexity analytics into a single accessible platform.

Proposed Contribution

The proposed project addresses the identified gap by developing an integrated, four-module algorithm-learning platform.

Main Contributions:

- Provides a step-by-step Algorithm Visualizer covering sorting, searching, graph, and DP algorithms.
- Provides an Algorithm Comparator that executes multiple algorithms on identical input and reports comparative metrics.
- Provides a Real-World Problem Solver that maps described problem scenarios to recommended algorithms with reasoning.
- Provides a Complexity Analyzer combining theoretical Big-O labels with empirically measured runtime and growth curves.
- Integrates all four modules behind a single, unified, responsive web dashboard with history logging.

Proposed Contribution Flow

Existing Single-Purpose Tools → Identified Gaps (no comparison, no problem mapping, disconnected complexity teaching) → Curated Algorithm & Problem Dataset → Visualizer + Comparator + Problem Solver + Complexity Analyzer → Unified Learning Dashboard

The proposed system therefore extends single-purpose algorithm visualization into a more complete, integrated algorithm-learning and analysis platform. It is intended to support and supplement, not replace, formal instructor-led DSA education.

CHAPTER 3

ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

Requirements

Table 1: Stakeholder / User Requirements

Stakeholder / User	Requirement
CS Student / Learner	Interactively visualize, compare, and understand algorithms and their real-world use.
Faculty / Instructor	Demonstrate algorithm behaviour live in class and assign comparative learning exercises.
Coding Bootcamp / Training Institute	Use a ready-made teaching resource covering visualization and complexity analysis.
Self-Learner / Competitive Programmer	Practice choosing the right algorithm for a given real-world problem.
System Administrator	Maintain the platform, algorithm library, and learner history data.

Table 2: Functional & Non-Functional Requirements

Requirement Type	Requirement	Measurable Target
Functional	Algorithm Visualizer	Animate step-by-step execution for sorting, searching, graph, and DP algorithms
Functional	Algorithm Comparator	Run 2-4 algorithms on identical input and report step count and execution time

Requirement Type	Requirement	Measurable Target
Functional	Real-World Problem Solver	Accept a problem description and recommend suitable algorithm(s) with reasoning
Functional	Complexity Analyzer	Display theoretical Big-O and empirically measured runtime/space growth curves
Functional	History logging	Store previous visualizations, comparisons, and analyses
Non-Functional	Usability	Simple, intuitive web-based learning dashboard
Non-Functional	Performance	Render visualizations smoothly for inputs up to the supported size range
Non-Functional	Reliability	Handle invalid/edge-case inputs without application failure
Non-Functional	Maintainability	Modular front-end/back-end implementation
Non-Functional	Portability	Run in modern desktop and tablet web browsers
Non-Functional	Accessibility	Colour-blind-safe visuals and keyboard-navigable interface
Non-Functional	Security	Sanitize custom input data to avoid injection in the comparator engine

Constraints & Applicable Standards

Table 3: Project Constraints

Constraint	Requirement / Effect
Browser Compute Limits	Very large input sizes may cause visualization lag or reduced frame rate.
Algorithm Coverage Scope	Limited to a curated set of classic sorting, searching, graph, and DP algorithms, not an exhaustive catalog.
Educational Accuracy	Visualizations must faithfully represent the real mechanics of each algorithm.
Device Diversity	The interface must render acceptably across varied screen sizes and browsers.
Time & Resource Constraints	Development is bounded by the academic project timeline and team resources.

Table 4: Applicable Standards / Practices

Standard / Code	Requirement	How Applied in This Project
ISO/IEC 25010:2023	Software product quality model	Considered usability, reliability, maintainability, performance and security during system design.
WCAG 2.1	Web content accessibility guidelines	Guided colour-contrast and keyboard-navigation choices for the visualization dashboard.
ISO/IEC 27001:2022	Information security management	Relevant principles considered for protecting learner history and usage data.
IEEE 1012-2016	Software verification and validation	Testing and validation activities applied to the developed modules and complete system.

Standard / Code	Requirement	How Applied in This Project
ACM/IEEE-CS Curricula Guidelines	DSA topic coverage reference	Referenced conceptually to align algorithm coverage with standard introductory/intermediate DSA curricula.

Design Specifications

Table 5: Design Specifications

Parameter	Target / Requirement	Unit	Priority
Visualization Frame Rate	Smooth step animation	fps	High
Supported Algorithm Categories	Sorting, searching, graph, DP	count	High
Comparator Concurrent Algorithms	Up to 4 algorithms side by side	count	Medium
Visualizer Input Size Range	5-100 elements	elements	High
Complexity Analyzer Metrics	Theoretical Big-O + empirical runtime/space	---	High
Problem-Mapping Coverage	Curated real-world scenario dataset	scenarios	Medium
Comparator Response Latency	Result available promptly for supported input sizes	ms	Medium
Interface	Responsive web dashboard	---	High

Alternative Solutions & Evaluation

Alternative 1: Static Textbook / Pseudocode Teaching

Relies entirely on textbook pseudocode and instructor explanation. Low cost and widely available, but conveys no dynamic behaviour, no comparison, and no measurable complexity feedback.

Alternative 2: Embedding an Existing Third-Party Visualizer

Embeds an existing single-purpose visualization tool (e.g., via an iframe) without added comparator, problem-mapping, or complexity-analysis features. Moderate effort, but no integration across modules, no comparative or applied context, and dependent on external site availability.

Alternative 3: Integrated AlgoVerse Platform (Proposed)

A custom-built platform combining a Visualizer, Comparator, Real-World Problem Solver, and Complexity Analyzer behind one dashboard, with a curated algorithm and problem-scenario dataset.

Evaluation Matrix (Scale: 1 = Low, 5 = High)

Table 6: Evaluation Matrix

Criterion	Weight	Alt 1: Textbook/Static	Alt 2: Embedded 3rd-Party Visualizer	Alt 3: AlgoVerse (Proposed)
Performance (learning value)	30%	2	3	5
Cost	15%	5	3	4
Learning Effectiveness	30%	2	3	5
Sustainability / Reusability	10%	3	3	4

Criterion	Weight	Alt 1: Textbook/Static	Alt 2: Embedded 3rd-Party Visualizer	Alt 3: AlgoVerse (Proposed)
Reliability	15%	4	4	4
Weighted Score	100%	2.75	3.15	4.55

The evaluation matrix gives the proposed AlgoVerse platform the highest weighted score of 4.55/5, making it the most suitable option among the considered alternatives.

Architecture Diagram

USER INPUT (Algorithm Selection / Custom Data / Problem Scenario Description)

| ▼

FRONTEND UI -- React-Based Dashboard + Visualization Engine

| ▼

MODULE 1: VISUALIZER | MODULE 2: COMPARATOR | MODULE 3: PROBLEM SOLVER | MODULE 4: COMPLEXITY ANALYZER

| ▼

ALGORITHM EXECUTION ENGINE -- Step Logger + Timer + Metric Collector

| ▼

RESULTS: Step Animation / Comparison Report / Recommended Algorithm / Complexity Report

| ▼

HISTORY & ANALYTICS STORE

Key Engineering Calculation

For the Complexity Analyzer module, each algorithm is benchmarked on a series of increasing input sizes n (e.g., 10, 50, 100, 500, 1000, 5000). For each n , execution time $T(n)$ is measured, averaged over multiple repeated runs to reduce measurement noise. The empirical order of growth

is estimated from the slope of $\log(T(n))$ plotted against $\log(n)$: a near-linear slope indicates approximately $O(n)$ growth, a slope near 2 indicates approximately $O(n^2)$ growth, and so on. This measured growth is displayed alongside the algorithm's theoretical Big-O label so that learners can visually compare theory against practice.

Systems Approach

The system is divided into interconnected subsystems: the algorithm and problem-scenario dataset, the visualization engine, the comparator engine, the problem-to-algorithm mapping engine, and the complexity-analysis engine. The dataset subsystem supplies structured algorithm metadata (pseudocode, category, theoretical complexity) to all other modules. The visualization and comparator engines share a common step-logging execution core, while the complexity analyzer reuses the same execution core for empirical timing. All module outputs are surfaced through a unified dashboard and stored in the history subsystem.

Trade-Off Analysis

Table 7: Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Visualization technique	Off-the-shelf charting library only	Faster to implement	Limited step-level animation control	Custom canvas/SVG-based animation engine selected for precise step-level visual control.
Comparator scope	Pairwise comparison only	Simpler UI	Limits richer multi-algorithm insight	Multi-algorithm (up to 4) comparator selected for richer comparative learning despite added UI complexity.

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Problem-mapping approach	Machine-learning classifier	Potentially more flexible matching	Requires large labelled training data, less transparent reasoning	Rule-based curated mapping selected for transparency and reliability given limited training data.
Complexity analysis depth	Theoretical labels only	Simple to implement	Disconnected from measurable performance	Combined theoretical + empirical benchmarking selected for stronger learning value.
Execution environment	Server-side execution	Handles larger inputs	Higher infrastructure cost and latency	Client-side (browser) execution selected for responsiveness and offline-friendly use, within input-size limits.

Sustainability, Ethics, Safety, Risk & Security

Table 8: Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Reduced dependence on printed textbooks / travel for coaching	Centralized, reusable web-based learning resource	A low-hardware-footprint web application was selected over any dedicated hardware/lab setup.

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Ethics	Risk of over-reliance on recommended algorithm without understanding reasoning	Recommendation reasoning, mapping rules, and disclaimers	The system is positioned as a learning aid; explanations accompany every recommendation.
Health & Safety	No direct physical hazard; indirect concern of extended screen time	Software-only interactive tool	No significant physical safety hazard identified during normal use.
Security	Protection of learner history / usage data and comparator input sanitization	Access-control and input-validation practices	Comparator inputs are validated/sanitized; history data access restricted to the owning session/user.

Ethical Considerations

- Algorithm recommendations should not be presented as the single correct answer without explanation of reasoning.
- Learner usage/history data should not be used or shared without appropriate authorization.
- Users should be informed that empirical timing results depend on their device and browser conditions.
- The curated algorithm and problem datasets should be kept accurate and free of misleading complexity labels.
- Instructor guidance should remain central to formal grading and assessment, not the platform's recommendations.

Risk Register

Table 9: Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Inaccurate real-world problem-to-algorithm mapping	Medium	Medium	Medium	Curated mapping rules reviewed against known problem-algorithm pairs	Low
Browser performance degradation for large inputs	Medium	Medium	Medium	Input-size caps and optimized rendering	Low
Misinterpretation of visualization/recommendation as absolute proof	Medium	Medium	Medium	Clear disclaimers and reasoning shown alongside results	Low
Noisy empirical runtime measurements	Medium	Medium	Medium	Averaging multiple benchmark runs per algorithm	Low
Accessibility gaps for colour-blind users	Medium	Low	Low	Colour-blind-safe palette and non-colour cues	Low

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Loss of stored visualization/comparison history	Low	Medium	Low	Regular data backups	Low
Scope creep in algorithm coverage	Medium	Low	Low	Prioritized, curated algorithm roadmap	Low
System/software failure	Low	Medium	Low	Error handling, logging, and testing	Low

CHAPTER 4

IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

Development Approach & Resources

The project was developed using a modular web-application development approach. First, an algorithm reference dataset (pseudocode, category, theoretical complexity) and a real-world problem-scenario dataset were curated and organized. A shared execution engine was then built to log step-by-step algorithm state, which was used to power both the Visualizer and Comparator modules. The Real-World Problem Solver was implemented as a rule-based mapping engine over the curated problem dataset, and the Complexity Analyzer was implemented by combining the stored theoretical complexity labels with empirically benchmarked runtime measurements. Finally, all four modules were integrated behind a unified React-based dashboard, and individual modules and the complete application were tested using a range of algorithms, input sizes, and problem scenarios.

Table 10: Materials, Components and Resources

Resource	Purpose
Algorithm Reference Dataset	Pseudocode, category, and theoretical complexity for supported algorithms
Real-World Problem Scenario Dataset	Curated problem statements mapped to recommended algorithms
JavaScript / TypeScript	Core application programming
React.js	Frontend framework and dashboard UI
D3.js	Step-by-step visualization rendering
Node.js / Express.js	Backend API and algorithm execution engine
MongoDB / JSON Dataset Store	Storage of algorithm metadata, problem dataset, and history

Resource	Purpose
Chart.js / Recharts	Complexity growth-curve and comparison charts
VS Code	Development environment
Windows/Linux/Mac PC	Development and testing execution

Software Implementation & Modern Tools Used

The system was implemented as a Node.js/Express backend serving a React-based single-page dashboard. A selected algorithm and input (or a curated sample dataset) is passed to the execution engine, which logs each step for the Visualizer, times each algorithm for the Comparator, and produces benchmark measurements for the Complexity Analyzer. The Real-World Problem Solver matches a submitted problem description against the curated scenario dataset and returns a recommended algorithm with supporting reasoning.

The major implementation modules are:

- Dataset Module -- stores algorithm metadata, pseudocode, complexity labels, and the problem-scenario dataset.
- Algorithm Visualizer Module (Module 1) -- renders step-by-step animated execution for sorting, searching, graph, and DP algorithms.
- Algorithm Comparator Module (Module 2) -- executes multiple algorithms on identical input and compares steps and execution time.
- Real-World Problem Solver Module (Module 3) -- maps a described problem scenario to recommended algorithm(s) with reasoning.
- Complexity Analyzer Module (Module 4) -- computes/benchmarks theoretical and empirical time/space complexity and plots growth curves.
- History Module -- records previous visualizations, comparisons, and analyses.
- Web Interface Module -- provides the unified, responsive learning dashboard.

Table 11: Tools Used

Tool / Software	Purpose	Stage Used
JavaScript / TypeScript	Application and module programming	Development
React.js	Dashboard and visualization UI	Frontend Development
D3.js	Step-by-step animation rendering	Visualization
Node.js / Express.js	Backend API and execution engine	Backend Development
MongoDB / JSON Store	Algorithm/problem dataset and history storage	Data Layer
Chart.js / Recharts	Runtime and complexity charting	Analysis
Postman	API endpoint testing	Testing
VS Code	Coding and debugging	All Stages

Implementation Evidence

[Insert Screenshot 1: Project folder structure] [Insert Screenshot 2: Algorithm Visualizer running on a sample input] [Insert Screenshot 3: Algorithm Comparator dashboard showing side-by-side results]

Test Plan & Verification

Testing was planned to verify the major functional and performance requirements of the system. Each module was tested independently before performing complete system integration testing.

Table 12: Test Plan & Verification

Requirement	Test Method	Specification / Required Value	Acceptance Criterion	Achieved Value
Algorithm Visualizer Rendering	Run visualizer on curated sample inputs	Correct step animation generated	Steps match reference execution	Passed
Algorithm Comparator Execution	Run 2-4 algorithms on identical input	Comparable step/time metrics generated	Metrics consistent across repeated runs	Passed
Real-World Problem Mapping	Submit curated problem scenarios	Correct recommended algorithm returned	Matches curated expected mapping	Passed
Complexity Analyzer Calculation	Benchmark algorithms across increasing n	Empirical curve generated with Big-O label	Curve trend matches theoretical class	Passed
History Logging	Perform multiple visualization/comparison runs	Results stored and retrievable	Previous results displayed correctly	Passed
UI Responsiveness	Load dashboard on varied screen sizes	Layout adapts without breaking	Usable on desktop and tablet widths	Passed

Results Status

The developed system successfully performed the major algorithm-learning functions. The application accepted algorithm and problem input, generated step-by-step visualizations, produced

comparative metrics across multiple algorithms, and displayed complexity analysis results through the unified monitoring dashboard.

Sample Output

Table 13: Sample Output

Parameter	Sample Result
Algorithms Compared	Bubble Sort vs Quick Sort
Input Size (n)	1,000 elements
Bubble Sort Execution Time	842 ms
Quick Sort Execution Time	6 ms
Theoretical Complexity -- Bubble Sort	$O(n^2)$
Theoretical Complexity -- Quick Sort	$O(n \log n)$
Recommended Algorithm	Quick Sort
Recommendation Confidence	93%

Result Visualizations

Fig 1: System Architecture Flow (see Chapter 3, Architecture Diagram).

Fig 2: Comparator Runtime Comparison Chart -- shows execution-time bars across compared algorithms for a given input size. [Insert Chart]

Fig 3: Complexity Analyzer Growth Curve -- shows measured runtime versus input size overlaid on the theoretical Big-O curve. [Insert Chart]

Graphs to Include:

- Execution Time vs Input Size for compared algorithms

- Step-Count Comparison Bar Chart
- Big-O Growth Curve Overlay (Theoretical vs Empirical)
- Algorithm Recommendation Accuracy Chart
- Module Usage / History Timeline

Data Analysis & Engineering Judgment

The results demonstrate that the developed platform can visualize algorithm execution, generate comparative metrics across multiple algorithms, and produce complexity analysis that reasonably aligns measured runtime with theoretical growth classes. The Real-World Problem Solver's recommendation module improves the practical usefulness of the platform by connecting abstract algorithms to concrete problem scenarios.

The dashboard improves the interpretability of the results by presenting step animation, comparison charts, recommendation reasoning, and complexity curves together. However, observed empirical runtime values can vary depending on device performance and browser load, and recommendation quality depends on how closely a submitted problem description matches the curated scenario dataset.

The main engineering observation is that dataset breadth and curation quality directly influence both recommendation accuracy and the educational value of the complexity comparison. Therefore, the current prototype is suitable for demonstration and supplementary learning, while a larger and more diverse algorithm/problem dataset would be required for broader classroom deployment.

Comparison with Existing Solutions & Achievement of Objectives

Table 14: Achievement of Objectives

Objective	Evidence	Achievement
Design integrated learning-platform architecture	Complete four-module application architecture	Fully Achieved

Objective	Evidence	Achievement
Curate algorithm reference dataset	Dataset covering sorting/searching/graph/DP algorithms	Fully Achieved
Implement Algorithm Visualizer	Working step-by-step animation engine	Fully Achieved
Implement Algorithm Comparator	Multi-algorithm side-by-side comparison engine	Fully Achieved
Implement Real-World Problem Solver	Problem-to-algorithm mapping module	Fully Achieved
Implement Complexity Analyzer	Theoretical + empirical complexity module	Fully Achieved
Develop unified web dashboard	Integrated React-based dashboard	Fully Achieved
Maintain history logging	History module storing past runs	Fully Achieved
Evaluate system through testing	Module and integration test results	Fully Achieved

Strengths & Limitations

Strengths

- Unified platform combining visualization, comparison, real-world mapping, and complexity analysis.
- Step-by-step animated visualization improves comprehension over static pseudocode.
- Side-by-side comparator directly demonstrates algorithm efficiency differences.
- Real-world problem mapping connects abstract algorithms to practical use cases.
- Combined theoretical and empirical complexity analysis reinforces Big-O understanding.

- History logging supports review and self-paced revision.
- Modular and extendable implementation.

Limitations

- Algorithm and problem-scenario coverage is curated, not exhaustive.
- Very large input sizes may reduce visualization smoothness in the browser.
- Empirical timing results depend on client device and browser conditions.
- Problem-to-algorithm mapping accuracy depends on scenario dataset quality.
- The system does not execute arbitrary user-submitted source code.
- Recommendations should not replace deeper instructor-led understanding of underlying reasoning.

Project Planning, Roles & Budget

Table 15: Project Milestone Timeline

Phase	Activity	Status
Phase 1	Problem identification and literature survey	Completed
Phase 2	Algorithm dataset and problem-scenario dataset curation	Completed
Phase 3	Algorithm Visualizer module development	Completed
Phase 4	Algorithm Comparator module development	Completed
Phase 5	Real-World Problem Solver module development	Completed
Phase 6	Complexity Analyzer module development	Completed
Phase 7	Unified dashboard integration	Completed
Phase 8	History logging module	Completed
Phase 9	Testing and evaluation	Completed
Phase 10	Documentation and final presentation	Completed

Table 16: Roles / Contribution

Student	Assigned Responsibility	Actual Contribution
K. Hemanth (192572074)	Module 1 -- Algorithm Visualizer	Algorithm Visualizer implementation, testing, and documentation.
G. Durga Surya Prasad (192425158)	Module 2 -- Algorithm Comparator	Algorithm Comparator implementation, testing, and documentation.
P. Dharani Govardhan (192525280)	Module 3 -- Real-World Problem Solver	Real-World Problem Solver implementation, testing, and documentation.
N. Kameshwar Reddy (192525272)	Module 4 -- Complexity Analyzer	Complexity Analyzer implementation, testing, and documentation.
P. Manoj (192411081)	Overall System Integration & Final Report	Overall System Integration & Final Report implementation, testing, and documentation.

Table 17: Project Budget

Item	Estimated Cost	Actual Cost
Existing Computer/Laptop	₹0	₹0
JavaScript / Node.js / Open-source Libraries	₹0	₹0
React.js	₹0	₹0
D3.js / Chart.js	₹0	₹0
MongoDB (local/free tier)	₹0	₹0

Item	Estimated Cost	Actual Cost
Algorithm/Problem Dataset (curated/open)	₹0	₹0
VS Code	₹0	₹0
Total	₹0	₹0

CHAPTER 5

CONCLUSION, FUTURE WORK AND REFLECTION

Conclusion

The project "AlgoVerse: An Interactive Platform for Visualizing, Comparing, and Learning Algorithms" was developed to address the limitations of static, single-purpose approaches to algorithm education. The proposed platform integrates step-by-step visualization, multi-algorithm comparison, real-world problem-to-algorithm mapping, and combined theoretical/empirical complexity analysis into a single, unified learning tool.

The developed web-based application successfully integrates algorithm and problem-dataset curation, an animated Visualizer, a metrics-driven Comparator, a rule-based Real-World Problem Solver, and a Complexity Analyzer, all behind a unified dashboard with history logging. The implemented testing confirmed the functioning of the major system modules and demonstrated the feasibility of combining visualization, comparison, applied problem-mapping, and complexity analytics to support intuitive algorithm learning.

Overall, the project provides a practical and extendable prototype that combines algorithm engineering, data visualization, and web technologies for algorithm education. The platform can support deeper learner understanding of algorithm behaviour and trade-offs, while continuing to rely on instructor guidance and formal coursework for complete mastery of the underlying theory.

Future Work

The following improvements can be considered for future development:

- Expand the algorithm library to include advanced graph algorithms, string algorithms, and introductory machine-learning algorithms.
- Support user-submitted custom code for visualization within a securely sandboxed execution environment.
- Add adaptive, personalized learning paths based on individual learner progress and history.
- Introduce gamification elements such as quizzes, challenges, and achievement tracking.
- Develop a mobile-friendly or native mobile application version of the platform.

- Integrate with learning-management systems (e.g., Moodle, Google Classroom) for classroom-scale deployment.
- Add collaborative/classroom mode allowing instructors to run live, shared visualizations.
- Support multi-language pseudocode display (e.g., Python, Java, C++) alongside visualizations.
- Deploy the platform on cloud infrastructure for wider, multi-institution accessibility.
- Conduct formal usability studies and classroom trials to validate educational effectiveness.

Professional Learning & Individual Reflection

New Knowledge Acquired

- Learned techniques for step-by-step algorithm visualization and animation.
- Gained practical knowledge of designing multi-algorithm comparison engines.
- Learned how to design and curate a real-world problem-to-algorithm mapping dataset.
- Understood the relationship between theoretical Big-O complexity and empirically measured performance.
- Learned full-stack web development using React, Node.js, and Express.
- Learned to integrate visualization, comparison, mapping, and analytics into one cohesive application.
- Gained experience in structured testing and system integration.

Engineering & Professional Skills Developed

- JavaScript/TypeScript programming and debugging.
- Data visualization design using D3.js and charting libraries.
- Algorithm analysis and benchmarking.
- Web application development using React and Node.js/Express.
- Software integration and modular development.
- Testing and performance analysis.
- Problem-solving and troubleshooting.

- Technical documentation and report preparation.

REFERENCES

1. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA, USA: MIT Press, 2022.
2. R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA, USA: Addison-Wesley, 2011.
3. S. Halim and F. Halim, "VisuAlgo: Visualizing data structures and algorithms through animation," National University of Singapore. [Online]. Available: <https://visualgo.net/>
4. D. Galles, "Data Structure Visualizations," University of San Francisco. [Online]. Available: <https://www.cs.usfca.edu/~galles/visualization/>
5. ACM/IEEE-CS Joint Task Force on Computing Curricula, "Computer Science Curricula," Association for Computing Machinery. [Online]. Available: <https://www.acm.org/education/curricula-recommendations>
6. W3C, "Web Content Accessibility Guidelines (WCAG) 2.1," World Wide Web Consortium. [Online]. Available: <https://www.w3.org/TR/WCAG21/>

Software / Technology References

7. React, "React Documentation," Meta. [Online]. Available: <https://react.dev/>
8. D3.js, "D3 Documentation." [Online]. Available: <https://d3js.org/>
9. OpenJS Foundation, "Node.js Documentation." [Online]. Available: <https://nodejs.org/>
10. MongoDB, "MongoDB Documentation." [Online]. Available: <https://www.mongodb.com/docs/>
11. Chart.js, "Chart.js Documentation." [Online]. Available: <https://www.chartjs.org/>
12. Mozilla, "MDN Web Docs." [Online]. Available: <https://developer.mozilla.org/>

APPENDICES

Appendix A -- Detailed Calculations

Benchmark Measurement Calculations

- Input size progression used for benchmarking (e.g., $n = 10, 50, 100, 500, 1000, 5000$)
- Mean and standard deviation of measured execution time across repeated runs
- Empirical order-of-growth estimate from the slope of $\log(T(n))$ vs $\log(n)$
- Step-count comparison across compared algorithms for identical input

Recommendation Calculation

Includes the calculation/logic used for:

- Keyword/feature match score between a submitted problem description and curated scenarios
- Recommended algorithm selection and confidence score

Example:

Bubble Sort Time = 842 ms, Quick Sort Time = 6 ms, Input Size = 1,000, Theoretical Complexity (Bubble) = $O(n^2)$, Theoretical Complexity (Quick) = $O(n \log n)$, Recommended Algorithm = Quick Sort, Recommendation Confidence = 93%.

Appendix B -- Engineering Drawings / Schematics

Overall System Architecture

(see Chapter 3, Architecture Diagram)

System Flowchart:

Algorithm/Problem Input → Frontend Dashboard → Execution Engine → Visualizer / Comparator / Problem Mapper / Complexity Analyzer → Results Display → Save History → End

Start → Select Algorithm or Describe Problem → Validate Input → Route to Relevant Module(s) → Execute & Log Steps/Timing → Compute Comparison / Recommendation / Complexity → Display Result → Save History → End

Module Architecture

- Module 1 -- Algorithm Visualizer
- Module 2 -- Algorithm Comparator
- Module 3 -- Real-World Problem Solver
- Module 4 -- Complexity Analyzer

Appendix C -- Source Code / Code Repository Information

The complete source code is maintained separately in the project repository. Only essential code excerpts should be included in the main report.

Project Structure

AlgoVerse/

```
|— server/
|   |— app.js
|   |— routes/
|   |— algorithms/
|   |   |— sorting/
|   |   |— searching/
|   |   |— graph/
|   |   └— dp/
|   |— comparator.js
|   |— problemSolver.js
|   └— complexityAnalyzer.js
|— client/
|   |— src/
```



```
| | | — components/
| | | — pages/
| | | — App.jsx
| | — public/
| — dataset/
| — algorithms.json
| — problem_scenarios.json
| — outputs/
| — history/
| — reports/
| — package.json
| — README.md
```

Repository

GitHub Repository: TEAM5-DAA/AlgoVerse

Implementation Folder:

Capstone Project/Implementation -- Include screenshots showing:

- GitHub repository
- Source-code files
- Dataset files
- Application running
- Final dashboard

Appendix D -- Datasheets

Since this is primarily a software/web-application project, include technical documentation for the main software components used.

React.js

- Dashboard and UI component development
- State management for visualizer/comparator views

D3.js

- Step-by-step animation rendering
- Complexity growth-curve plotting

Node.js / Express.js

- Backend API
- Algorithm execution and benchmarking engine

MongoDB

- Algorithm and problem-scenario dataset storage
- History logging

Appendix E -- Experimental / Raw Data

Dataset Information

Category	Purpose
Sorting Algorithms	Bubble, Selection, Insertion, Merge, Quick, Heap Sort samples
Searching Algorithms	Linear Search, Binary Search samples
Graph Algorithms	BFS, DFS, Dijkstra samples
Dynamic Programming Algorithms	Fibonacci, Knapsack, LCS samples
Real-World Problem Scenarios	Curated scenarios mapped to recommended algorithms

Sample Testing Data

Test ID	Input	Expected Result	Actual Result	Status
T01	Sort 100 random integers	Correctly sorted output	Correctly sorted output	Pass
T02	Compare Bubble Sort vs Quick Sort (n=1000)	Quick Sort faster	Quick Sort faster	Pass
T03	"Find shortest route between two cities" scenario	Recommend Dijkstra's Algorithm	Recommended Dijkstra's Algorithm	Pass
T04	Benchmark across n = 10 to 5000	Growth curve matches theoretical class	Growth curve matched expected class	Pass
T05	Empty / invalid input array	Graceful error message	Graceful error message shown	Pass

Also include:

- Comparator runtime comparison chart
- Complexity growth-curve chart (empirical vs theoretical)
- Recommendation accuracy summary
- Screenshots of test outputs

Appendix F -- Ethics / Institutional Approval

Not Applicable: This project was developed as an academic software prototype using curated/open algorithm reference material and simulated real-world problem scenarios for research and educational demonstration purposes. No human-subject experimentation was conducted.

Appropriate care was taken to avoid presenting algorithm recommendations as an absolute, final engineering decision without further learner or instructor judgement.

If your college provided an approval/permission document, place a scanned copy here.

Appendix G -- Project Meeting / Progress Records

Include evidence of project development and review activities.

Date / Activity / Discussion / Outcome

Week 1 -- Project topic discussion -- Problem identified

Week 2 -- Literature survey -- Existing visualization tools studied

Week 3 -- Dataset curation -- Algorithm and problem-scenario datasets organized

Week 4 -- Visualizer & Comparator development -- Initial modules built

Week 5 -- Problem Solver & Complexity Analyzer development -- Modules implemented

Week 6 -- Dashboard integration -- Unified UI assembled

Week 7 -- Testing -- System tested across modules

Week 8 -- Final documentation -- Report and presentation prepared

Include photographs of:

- Review 1
- Review 2
- Team/project discussion
- System demonstration

Appendix H -- User / Industry Feedback

If applicable, include feedback received during project reviews or demonstrations.

Reviewer / User	Feedback	Action Taken
Faculty Reviewer	Improve comparator result visualization	Added side-by-side runtime bar chart
Faculty Reviewer	Explain reasoning behind algorithm recommendations	Added recommendation-reasoning panel
User Testing	Add history of past visualizations	Added history/bookmark logging module
User Testing	Improve complexity explanation	Added theoretical vs empirical growth-curve overlay

Appendix I -- Publications / Patents / Competitions / Product Outcomes

If applicable, include:

- Research publication
- Conference paper
- Hackathon participation
- Competition certificate
- Patent information
- Project demonstration
- Product/prototype outcome

Appendix J -- Individual Contribution Evidence

J. K. Hemanth -- 192572074

J. G. Durga Surya Prasad -- 192425158

J. P. Dharani Govardhan -- 192525280

J. N. Kameshwar Reddy -- 192525272

J. P. Manoj -- 192411081

Overall System Integration & Final Report Contribution (Report Submitter -- P. Manoj, 192411081):

- Integration of all four modules into a unified Flask/Node-based web application
- Complete system integration testing across modules
- Consolidated documentation and final report preparation